

Displacement, Worked Example by Example

Filling in $\text{disp}(x) = \sum_F(\dots)$ for relative clauses, adjectives, adverbs, determiners

Companion to `displacement_handout.tex` (`experiments/22_rasp.1_hi_hf`)

0. The one rule we fill in every time

Head-final (HF) is a permutation of head-initial (HI): each token keeps its identity and only moves. Its target slot is its start index plus a *displacement*, and the displacement is a **sum of one $\pm$ per flip node the token sits inside**:

The master formula

$$\text{hf_pos}(x) = \text{hi_pos}(x) + \text{disp}(x), \quad \text{disp}(x) = \sum_{\substack{F \text{ a flip node} \\ \text{whose span contains } x}} \begin{cases} +|\text{comp}(F)| & x \in \text{head}(F) \\ -|\text{head}(F)| & x \in \text{comp}(F) \end{cases}$$

Which nodes are **flips** (reverse their two children) versus **invariant** (keep child order) is fixed by the grammar:

Flip (head comp reverse)	Invariant (order kept)
CP_sent=[C S], CP_rel=[C_rel S_gap]	S=[subj VP], DP=[D NP]
VP=[V(+Adv) obj/clause]	S_obj_gap=[DP V], NP_singular_adj=[Adj N]
VP_*_adv=[V Adv], NP_singular=[noun rel. clause]	every <i>unary</i> node (one child)

How to read each example

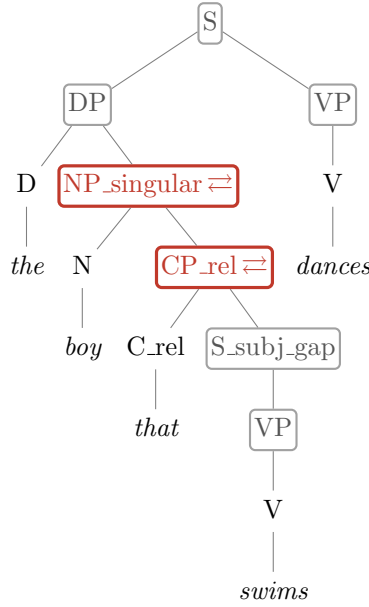
(1) draw the tree, tag **flip**/invariant nodes; (2) list each flip's |head|, |comp|; (3) for each token, add one term per enclosing flip ($+|\text{comp}|$ if it is in that flip's head, $-|\text{head}|$ if in its comp); (4) sum $\Rightarrow$ **disp**, then $\text{hf_pos} = i + \text{disp}$. In the tables, **green +** = a head contribution, **blue -** = a comp contribution.

1. Relative clauses (three types)

A relative clause attaches by the flip $\text{NP_singular}=[\text{core noun}|\text{CP_rel}]$: the whole clause jumps in *front* of the noun it modifies. Inside, $\text{CP_rel}=[\text{that}|\text{S_gap}]$ is a second flip. What differs between the three types is the *gap clause* **S_gap** – and, crucially, whether it contains any *further* flip.

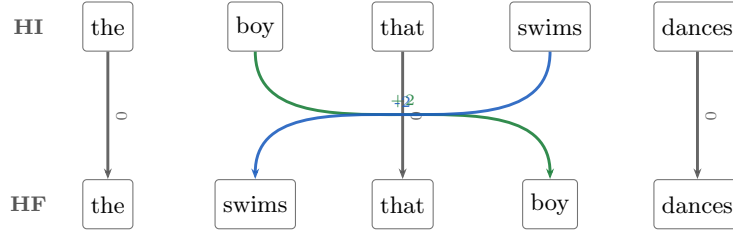
1a. Subject-gap, intransitive – *the boy that swims dances*

HI: the boy that swims dances $\rightarrow$ **HF**: the swims that boy dances



Flip nodes. NP_singular: |head|=1 (*boy*), |comp|=2 (*that swims*). CP_rel: |head|=1 (*that*), |comp|=1 (*swims*). The gap clause S_subj_gap→VP→*swims* is a chain of *unary* (invariant) nodes – no flip inside.

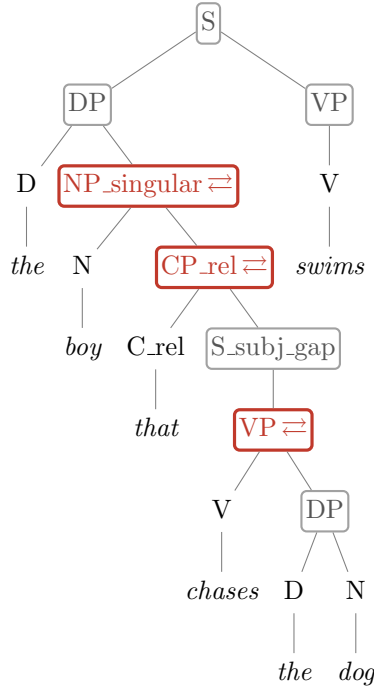
tok (<i>i</i>)	disp = $\sum_F(\dots)$	=disp	hf
the (0)	(no enclosing flip)	0	0
boy (1)	+2 (NP.head)	+2	3
that (2)	-1 (NP.comp) +1 (CPrel.head)	0	2
swims (3)	-1 (NP.comp) -1 (CPrel.comp)	-2	1
dances (4)	(no enclosing flip)	0	4



Takeaway. *that* lands back on 0: its +1 as CP_rel head exactly cancels its -1 as NP comp. *swims* (the whole gap clause) collects two -1's and slides left of the noun it modifies.

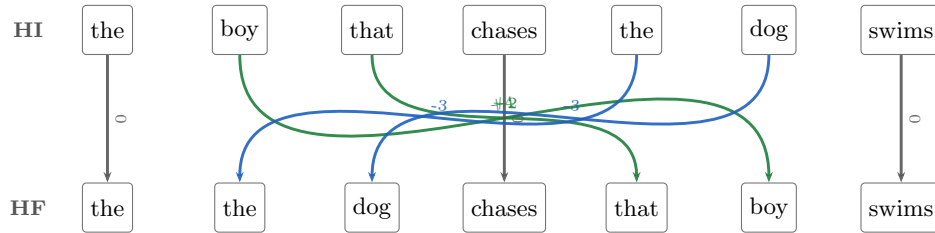
1b. Subject-gap, transitive – *the boy that chases the dog swims*

HI: the boy that chases the dog swims → **HF:** the the dog chases that boy swims



Flip nodes. NP_singular: |head|=1, |comp|=4 (*that chases the dog*). CP_rel: |head|=1, |comp|=3 (*chases the dog*). VP: |head|=1 (*chases*), |comp|=2 (*the dog*). The subject gap holds a *transitive* VP, so a **third** flip appears inside.

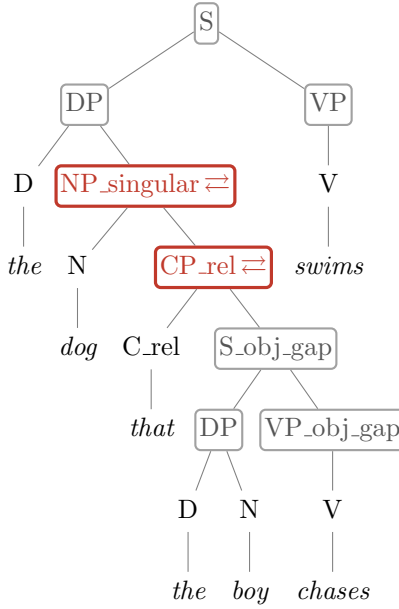
tok (<i>i</i>)	disp = $\sum_F(\dots)$	=disp	hf
the (0)	(no enclosing flip)	0	0
boy (1)	+4 (NP.head)	+4	5
that (2)	-1 (NP.comp) +3 (CPrel.head)	+2	4
chases (3)	-1 (NP.comp) -1 (CPrel.comp) +2 (VP.head)	0	3
the (4)	-1 (NP.comp) -1 (CPrel.comp) -1 (VP.comp)	-3	1
dog (5)	-1 (NP.comp) -1 (CPrel.comp) -1 (VP.comp)	-3	2
swims (6)	(no enclosing flip)	0	6



Takeaway. *chases* is a flip **head** (+2) yet ends stationary — the two outer -1's cancel its +2. The object *the dog* is buried under *three* comps, so it takes the full -3.

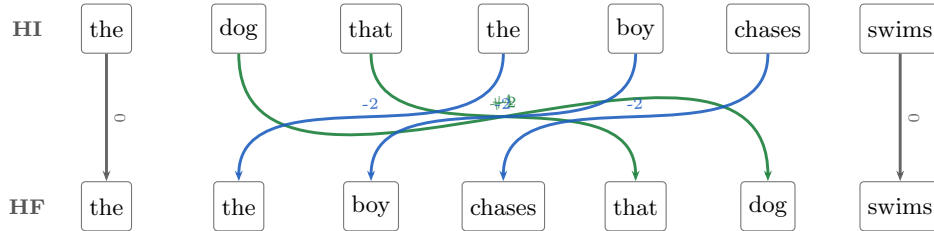
1c. Object-gap — *the dog that the boy chases swims*

HI: the dog that the boy chases swims → **HF:** the the boy chases that dog swims



Flip nodes. NP_singular: |head|=1, |comp|=4 (*that the boy chases*). CP_rel: |head|=1, |comp|=3 (*the boy chases*). Now the gap clause is S_obj_gap=[DP|V], which is **invariant**, and VP_obj_gap is a bare unary verb – so unlike 1b there is **no third flip**.

tok (<i>i</i>)	disp = $\sum_F(\dots)$	=disp	hf
the (0)	(no enclosing flip)	0	0
dog (1)	+4 (NP.head)	+4	5
that (2)	-1 (NP.comp) +3 (CPrel.head)	+2	4
the (3)	-1 (NP.comp) -1 (CPrel.comp)	-2	1
boy (4)	-1 (NP.comp) -1 (CPrel.comp)	-2	2
chases (5)	-1 (NP.comp) -1 (CPrel.comp)	-2	3
swims (6)	(no enclosing flip)	0	6

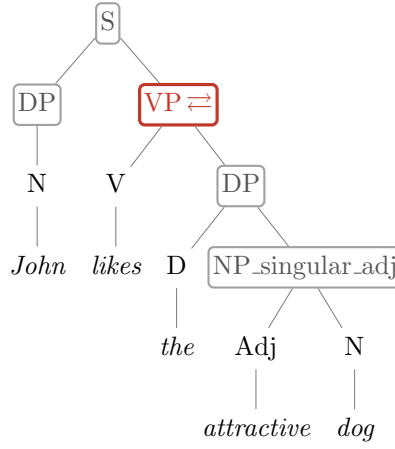


Subject-gap vs. object-gap: the one row that differs

Compare *chases* across 1b and 1c. In 1b (subject gap) it heads a transitive VP flip, so it gets an extra +2 and nets 0. In 1c (object gap) the verb sits in an *invariant* S_obj_gap with its object already relativized away – *no verb flip* – so it only inherits the two -1 pushes and nets -2. Same two outer flips, different *inside*: object gaps keep “the boy chases” rigid; subject gaps reverse “chases | the dog”.

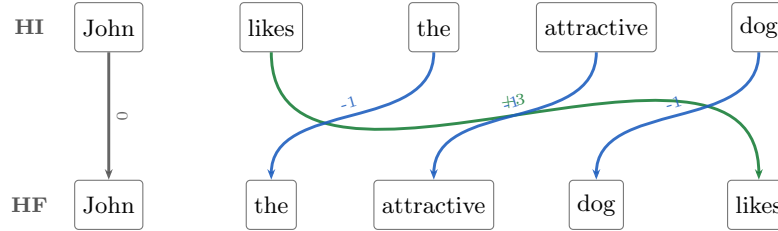
2. Adjectives – *John likes the attractive dog*

HI: John likes the attractive dog → **HF:** John the attractive dog likes



Flip nodes. VP: `|head|=1` (*likes*), `|comp|=3` (*the attractive dog*). The adjective’s node `NP_singular_adj` is **invariant**: an adjective *never* reorders with its noun, and only rides along inside the enclosing flip.

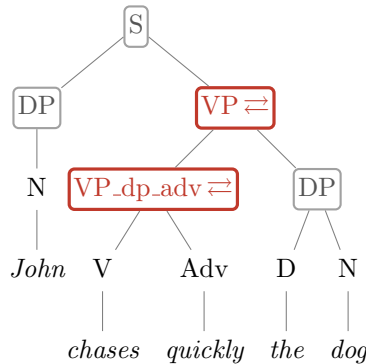
tok (<i>i</i>)	disp = $\sum_F(\dots)$	=disp	hf
John (0)	(no enclosing flip)	0	0
likes (1)	+3 (VP.head)	+3	4
the (2)	−1 (VP.comp)	−1	1
attractive (3)	−1 (VP.comp)	−1	2
dog (4)	−1 (VP.comp)	−1	3



Takeaway. *the attractive dog* moves as one rigid block: all three tokens get the *same* −1 (the VP comp), so their internal order – determiner, adjective, noun – is preserved. An adjective adds nothing new to the displacement rule; it only makes the complement it lives in one token *larger*, which shows up as a bigger `+|comp|` on the verb.

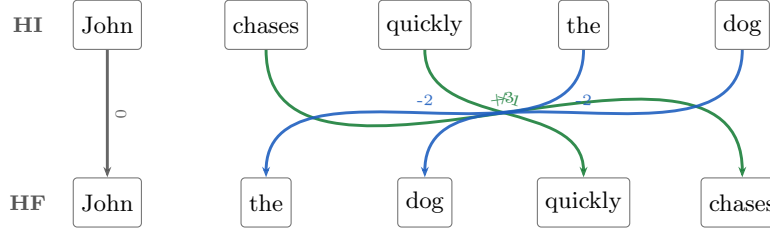
3. Adverbs – *John chases quickly the dog*

HI: John chases quickly the dog → **HF:** John the dog quickly chases



Flip nodes. VP: head block = VP_dp_adv (*chases quickly*, |**head**|=2), comp = *the dog* (|**comp**|=2).
 VP_dp_adv: |**head**|=1 (*chases*), |**comp**|=1 (*quickly*). An adverb sits in **two** flips at once: the *local* verb–adverb swap and the outer VP block-swap.

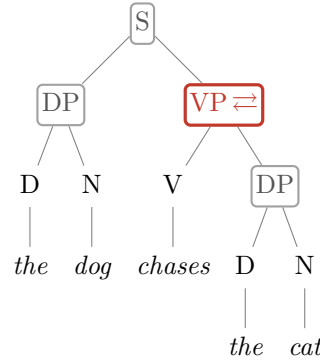
tok (<i>i</i>)	disp = $\sum_F(\dots)$	=disp	hf
John (0)	(no enclosing flip)	0	0
chases (1)	+2(VP.head) +1(Vadv.head)	+3	4
quickly (2)	+2(VP.head) -1(Vadv.comp)	+1	3
the (3)	-2(VP.comp)	-2	1
dog (4)	-2(VP.comp)	-2	2



Takeaway. *quickly* is the **head** of the outer VP block (+2, it jumps over *the dog* with the verb) but the **comp** of the inner VP_dp_adv (-1, it falls behind the verb locally). Net +1. The verb *chases* is a head of *both* flips, so it collects +2+1 = +3. Note the object’s push is now -2, not -1, because the head block it jumps back over has *size 2* (verb + adverb).

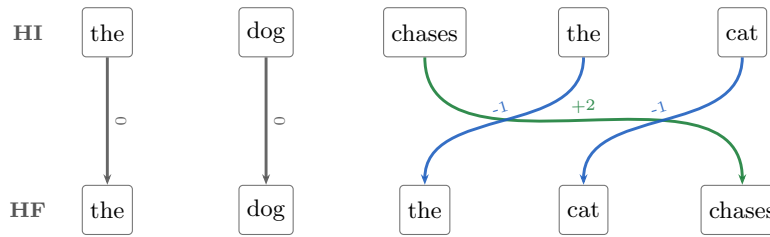
4. Determiners – *the dog chases the cat*

HI: the dog chases the cat → **HF:** the dog the cat chases



Flip nodes. VP: |**head**|=1 (*chases*), |**comp**|=2 (*the cat*). Both DP=[D|N] nodes are **invariant**: a determiner never swaps with its noun.

tok (<i>i</i>)	disp = $\sum_F(\dots)$	=disp	hf
the (0)	(subject DP; no enclosing flip)	0	0
dog (1)	(subject DP; no enclosing flip)	0	1
chases (2)	+2(VP.head)	+2	4
the (3)	-1(VP.comp)	-1	2
cat (4)	-1(VP.comp)	-1	3



Takeaway. The two determiners behave oppositely *only* because of what encloses them, never because of any determiner-specific rule. The *subject* *the*(0) sits under an invariant **S/DP** with no flip above it $\Rightarrow$ it does not move. The *object* *the*(3) is inside the VP comp $\Rightarrow$ it gets the same -1 as its noun and stays glued in front of *cat*.

A determiner subtlety, seen back in Section 1

In the relative-clause examples the head DP is $\text{DP}=[\text{the}|\text{NP_singular}]$. The flip is $\text{NP_singular}=[\text{noun}|\text{CP_rel}]$ – and the determiner *the* is a *sibling* of that flip, *not inside* it. That is exactly why *the*(0) stays at 0 while *boy/dog*(1) jumps $+4$: only the noun is the flip’s head; its determiner is left behind. “Contains” in the master formula is literal.

Every HI/HF pair and every `disp/hf_pos` column above is machine-verified against `grammar-oracle.py` (see `_verify_examples.py`).